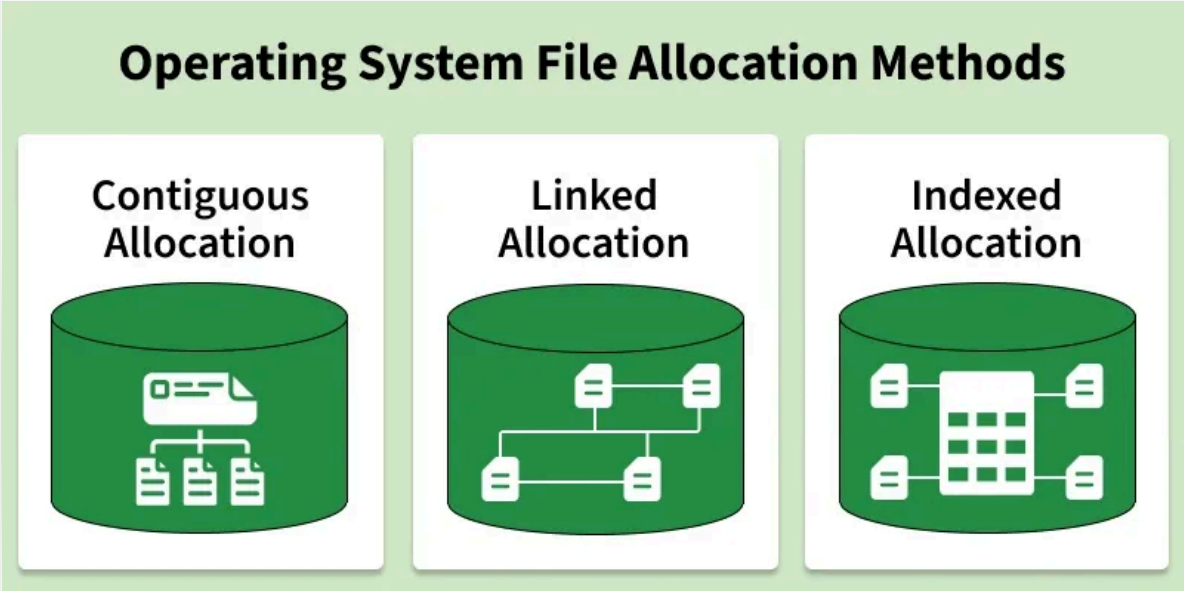
Last Minute Notes – Operating Systems

Last Updated : 9 Feb, 2026

An Operating System (OS) is a system software that manages computer hardware, software resources, and provides common services for computer programs. It acts as an interface between the user and the computer hardware.



6 / 7

Types of Operating System (OS):

Batch OS

- Executes a set of similar jobs together.
- CPU processes one job at a time; next job starts only after the previous finishes.

Multiprogramming OS

- Multiple jobs are loaded in memory.
- CPU switches to another job when one is waiting (e.g., for I/O).
- Keeps CPU always busy.

Time-Sharing / Multitasking OS

- A type of multiprogramming where each process gets a fixed time slice (quantum).
- CPU switches between tasks quickly so all users/processes feel simultaneous execution.

Multiprocessing OS

- Uses more than one CPU for executing processes.
- Improves system throughput.

Multi-user OS

- Allows multiple users to work at the same time.
- Can run on single or multiple processors.

Distributed OS

- Manages a group of interconnected computers that communicate over a network.
- Each system has its own CPU and memory, but works as a single system.

Network OS

- Runs on a server to manage files, users, security, and applications over a network.
- Allows shared access to resources like printers and files.

Real-Time OS (RTOS)

- Designed for dedicated tasks with strict deadlines.
- Common in embedded systems, medical devices, and industrial machines.

Threads

A lightweight process and the basic unit of CPU execution.

Features:

- Has its own program counter, registers, and stack.
- Shares code, data, files, and signals with other threads of the same process.

Creation : Using `fork()` system call.

- A process with n fork() calls creates $2^n - 1$ child processes.

Types of Threads:

Based on number:

- Single-threaded process
- Multi-threaded process

Based on level:

- User-level threads
- Kernel-level threads

Examples: Java Threads, POSIX Threads, Windows Threads, Solaris Threads

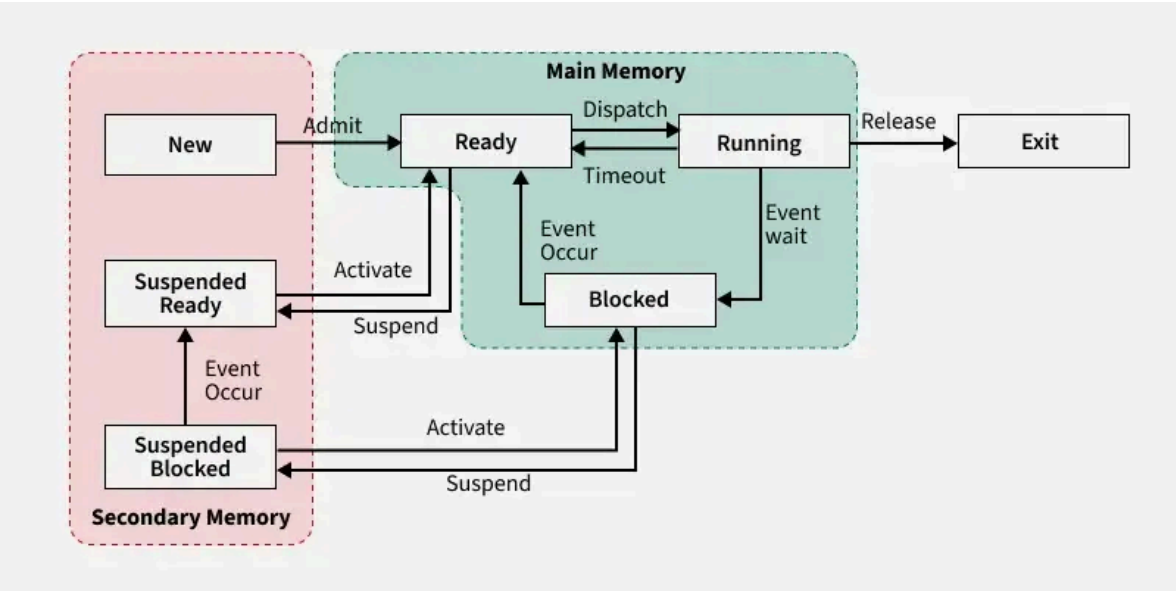
Process

Process is a program in execution.

Key Points:

- Program Counter (PC) points to the next instruction to execute.

- Represented internally by Process Control Block (PCB).
- Can contain one or more threads.



Process States Diagram

Schedulers

The OS uses schedulers to manage which process gets CPU time.

Long-term Scheduler

- Controls admission of new processes into main memory.
- Transitions processes from New → Ready state.

Medium-term Scheduler

- Temporarily suspends or resumes processes.
- Performs swap-out (main → secondary memory) and swap-in (secondary → main memory).

Short-term Scheduler

- Selects a process from the ready queue for execution.
- Transitions processes from Ready → Running state.

Dispatchers

Responsible for loading the selected process onto the CPU.

Performs context switching:

- Saves the state of the current process.
- Loads the state of the next process to execute.

CPU Scheduling Algorithms

Need of Scheduling

- Processes involve CPU time and I/O time.
- In uniprogramming, CPU sits idle during I/O waits.
- Multiprogramming allows CPU to execute another process while one waits for I/O.

- Scheduling decides which process gets the CPU and when.

Process Timing Terms

Arrival Time (AT)

- When the process enters the ready queue.

Completion Time (CT)

- When the process finishes execution.

Burst Time (BT)

- Time needed by CPU to execute the process.

Turnaround Time (TAT)

- Total time process spends in the system.
- Formula: TAT=CT−AT

Waiting Time (WT)

- Total time process waits in the ready queue.
- Formula : WT=TAT−BT

Objectives of Process Scheduling Algorithm:

- Max CPU utilization (Keep CPU as busy as possible)
- Fair allocation of CPU.
- Max throughput (Number of processes that complete their execution per time unit)
- Min turnaround time (Time taken by a process to finish execution)
- Min waiting time (Time for which a process waits in ready queue)
- Min response time (Time when a process produces first response)

Different CPU Scheduling Algorithms:

1. First Come First Serve (FCFS): First Come, First Serve (FCFS) is one of the simplest types of CPU scheduling algorithms. It is exactly what it sounds like: processes are attended to in the order in which they arrive in the ready queue, much like customers lining up at a grocery store.

- It is a non pre-emptive algorithm.
- Processes are assigned to the CPU based on their arrival times. If two or more processes have the same arrival times, then the processes are assigned based on their process ids.
- It is free from starvation

2. Shortest Job First (SJF): Shortest Job First (SJF) or Shortest Job Next (SJN) is a scheduling process that selects the waiting process with the smallest execution time to execute next.

- It is a non-pre-emptive algorithm.
- If all the processes have the same burst times, SJF behaves as FCFS.
- In SJF, the burst times of all the processes should be known prior execution.
- It minimizes the average response time of the processes.
- There is a chance of starvation.

3. Shortest Remaining Time First (SRTF): It is preemptive mode of SJF algorithm in which jobs are scheduled according to the shortest remaining time.

- It is a pre-emptive algorithm.
- Processes are assigned to the CPU based on their shortest remaining burst times.
- If all the processes have the same arrival times, SRTF behaves as SJF.
- It minimizes the average turnaround time of the processes.
- If shorter processes keep on arriving, then the longer processes may starve.

4. Round Robin (RR) Scheduling: It is a method used by operating systems to manage the execution time of multiple processes that are competing for CPU attention.

- RR scheduling is a pre-emptive FCFS based on the concept of time quantum or time slice.
- If the time quantum is too small, then the number of context switches (overhead) will increase and the average response time will decrease.
- If the time quantum is too large, then the number of context switches (overhead) will decrease and the average response time will increase.
- If the time quantum is greater than the burst times of all the processes, RR scheduling behaves as FCFS.

5. Priority Based scheduling: In this scheduling, processes are scheduled according to their priorities, i.e., highest priority process is schedule first. If priorities of two processes match, then scheduling is according to the arrival time.

- Priority scheduling can either be pre-emptive or non-pre-emptive.
- In pre-emptive priority scheduling, a process is voluntarily pre-empted whenever a higher priority process arrives.
- In non-pre-emptive priority scheduling, the scheduler picks up the highest priority process.
- If all the processes have equal priority, then priority scheduling behaves as FCFS.

6. Highest Response Ratio Next (HRRN): In this scheduling, processes with highest response ratio is scheduled. This algorithm avoids starvation.

- It is a non pre-emptive algorithm.
- Processes are assigned to the CPU based on their highest response ratio.
- Response ratio is calculated as:

$$\text{Response Ratio} = \frac{WT+BT}{BT}$$

- It is free from starvation.
- It favors the shorter jobs and limits the waiting time of the longer jobs.

Some useful facts about Scheduling Algorithms:

1. FCFS can cause long waiting times, especially when the first job takes too much CPU time.
2. Both SJF and Shortest Remaining time first algorithms may cause starvation. Consider a situation when a long process is there in the ready queue and shorter processes keep coming.
3. If time quantum for Round Robin scheduling is very large, then it behaves same as FCFS scheduling.
4. SJF is optimal in terms of average waiting time for a given set of processes. SJF gives minimum average waiting time, but problems with SJF is how to know/predict the time of next job.

Critical Section Problem

1. **Critical Section -** The portion of the code in the program where shared variables are accessed and/or updated.
2. **Remainder Section -** The remaining portion of the program excluding the Critical Section.
3. **Race around Condition -** The final output of the code depends on the order in which the variables are accessed. This is termed as the race around condition.

Requirements for a Correct Solution

A solution for the critical section problem must satisfy the following three conditions:

Mutual Exclusion

- If process Pi is in its CS, no other process can enter its CS.

Progress

- If no process is in the CS, selection of the next process to enter CS must not be postponed indefinitely.

Bounded Waiting

- A process should not wait forever to enter the CS.
- There must be a limit on how many times others can enter before it.

Synchronization Tools

Semaphore

A semaphore is a synchronization tool used by the OS to control access to shared resources in a concurrent system.

- It is an integer variable
- Can be accessed only through atomic operations
- Prevents race conditions

Atomic Operations

Operations that execute in one indivisible CPU step:

- No pre-emption
- No interference by other processes

Two Atomic Operations:

- wait() // also called P()
- signal() // also called V()

Types of Semaphores:

Counting Semaphore

- Value can range over any integer.
- Used when multiple instances of a resource exist.

Mutex

- Used to provide mutual exclusion.
- Only one thread/process can enter the CS at a time.
- Common in producer–consumer problems.

Misconception: There is an ambiguity between binary semaphore and mutex. We might have come across that a mutex is binary semaphore. But they are not! The purpose of mutex and semaphore are different. May be, due to similarity in their implementation a mutex would be referred as binary semaphore.

Deadlock

Deadlock is a situation where a set of processes are permanently blocked because:

- Each process holds at least one resource, and
- Each process is waiting for a resource held by another process.

Necessary Conditions for Deadlock

- **Mutual Exclusion -** One or more than one resource are non-sharable (Only one process can use at a time).
- **Hold and Wait -** A process is holding at least one resource and waiting for resources.
- **No Preemption -** A resource cannot be taken from a process unless the process releases the resource.
- **Circular Wait -** A set of processes are waiting for each other in circular form.

Deadlock handling

There are three ways to handle deadlock

Deadlock prevention

To ensure the system never enters a deadlock state, at least one of the conditions for deadlock must be prevented:

Mutual Exclusion: This condition cannot be removed because some resources (non-shareable resources) must be exclusively allocated to one process at a time.

Hold and Wait: This condition can be avoided using the following strategies:

- Allocate all required resources to a process before it starts execution. However, this approach might lead to low resource utilization.
- Force a process to release all its held resources before requesting new ones. However, this might cause **starvation** (a process waiting indefinitely).

Pre-emption:If a process P1P1P1 requests a resource RRR that is held by another process P2P2P2:

- If P2P2P2 is still running, P1P1P1 will have to wait.
- Otherwise, the resource RRR can be taken (pre-empted) from P2P2P2 and allocated to P1P1P1.

Circular Wait: This condition can be avoided using the following method:

- Assign a unique number to each resource (assuming there is only one instance of each resource).
- Ensure processes request resources in a strict order, either increasing or decreasing based on the assigned numbers.

Deadlock Avoidance

The Deadlock Avoidance Algorithm prevents deadlocks by monitoring resource usage and resolving conflicts before they occur, like rolling back processes or reallocating resources. It minimizes deadlocks but doesn't fully guarantee their prevention. The two key techniques are:

1. **Process Initiation Denial**: Blocks processes that might cause deadlock.

2. **Resource Allocation Denial**: Rejects resource requests that could lead to deadlock.

Banker's Algorithm

The Banker's Algorithm is a resource allocation and deadlock avoidance algorithm used in operating systems. It ensures that the system remains in a safe state by simulating the allocation of resources to processes without running into a deadlock.

The following Data structures are used to implement the Banker's Algorithm:

1. Available: It is a 1-D array of size **'m'** indicating the number of available resources of each type. Available[j] = k means there are **'k'** instances of resource type **R_j**.

2. Max: It is a 2-d array of size ' **n*m'** that defines the maximum demand of each process in a system. Max[i, j] = k means process **P_i** may request at most **'k'** instances of resource type **R_j**.

3.Allocated: It is a 2-d array of size **'n*m'** that defines the number of resources of each type currently allocated to each process. Allocation[i, j] = k means process **P_i** is currently allocated **'k'** instances of resource type **R_j**.

4. Need: It is a 2-d array of size **'n*m'** that indicates the remaining resource need of each process. Need [i, j] = k means process **P_i** currently needs **'k'** instances of resource type **R_j** Need [i, j] = Max [i, j] – Allocation [i, j].

Deadlock Detection and Recovery

Detection: A cycle in the resource allocation graph represents deadlock only when resources are of single-instance type. If the resources are of multiple instance type, then the safety algorithm is used to detect deadlock.

Recovery: A system can recover from deadlock through adoption of the following mechanisms:

- Process termination
- Resource Pre-emption

Deadlock Ignorance

In the Deadlock ignorance method the OS acts like the deadlock never occurs and completely ignores it even if the deadlock occurs. This method only applies if the deadlock occurs very rarely. The algorithm is very simple. It says, " if the deadlock occurs, simply reboot the system and act like the deadlock never occurred." That's why the algorithm is called the **Ostrich Algorithm**.

Memory Management

In multiprogramming system, the task of subdividing the memory among the various processes is called memory management. The task of the memory management unit is the efficient utilization of memory and minimize the internal and external fragmentation.

Requirement of Memory Management

- Allocate and de-allocate memory before and after process execution.
- To keep track of used memory space by processes.
- To minimize fragmentation issues.
- To proper utilization of main memory.
- To maintain data integrity while executing of process.

Logical and Physical Address Space

- **Logical Address Space:** An address generated by the CPU is known as a "Logical Address". It is also known as a Virtual address. Logical address space can be defined as the size of the process. A logical address can be changed.

- **Physical Address Space:** An address seen by the memory unit (i.e. the one loaded into the memory address register of the memory) is commonly known as a "Physical Address". A Physical address is also known as a Real address.

Static and Dynamic Loading

Loading a process into the main memory is done by a loader. There are two different types of loading :

- **Static Loading:** Static Loading is basically loading the entire program into a fixed address. It requires more memory space.

- **Dynamic Loading:** The entire program and all data of a process must be in physical memory for the process to execute. So, the size of a process is limited to the size of physical memory. To gain proper memory utilization, dynamic loading is used. In dynamic loading, a routine is not loaded until it is called.

Memory Management Techniques

(a) Single Partition Allocation Schemes - The memory is divided into two parts. One part is kept to be used by the OS and the other is kept to be used by the users.
(b) Multiple Partition Schemes -

1. **Fixed Partition -** The memory is divided into fixed size partitions.
2. **Variable Partition -** The memory is divided into variable sized partitions.

Variable partition allocation schemes:

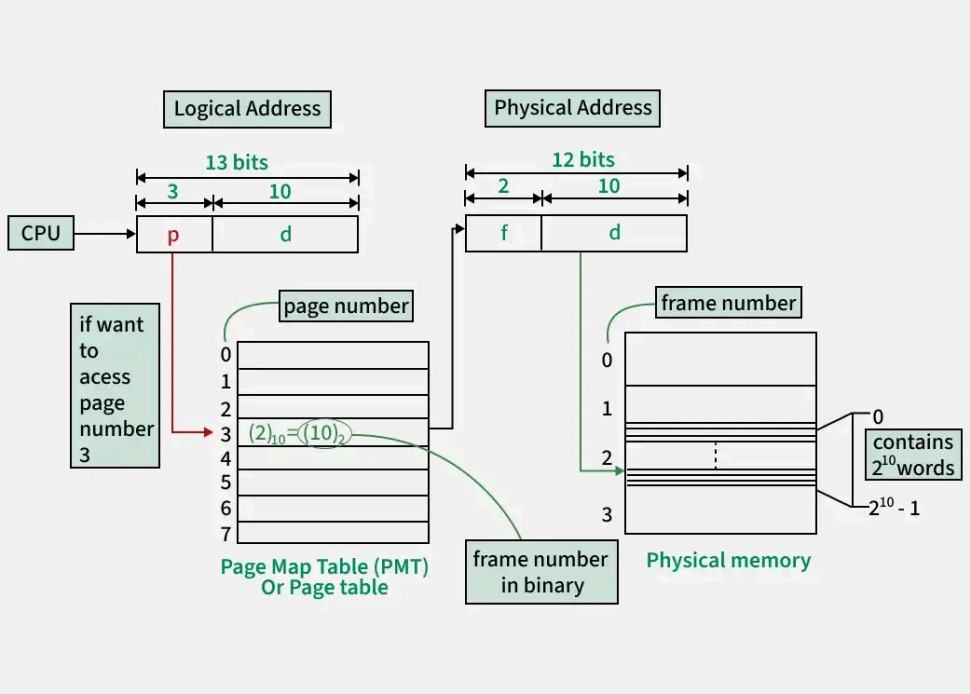
1. **First Fit -** The arriving process is allotted the first hole of memory in which it fits completely.
2. **Best Fit -** The arriving process is allotted the hole of memory in which it fits the best by leaving the minimum memory empty.
3. **Worst Fit -** The arriving process is allotted the hole of memory in which it leaves the maximum gap.

Note:

- Best fit does not necessarily give the best results for memory allocation.
- The cause of external fragmentation is the condition in Fixed partitioning and Variable partitioning saying that entire process should be allocated in a contiguous memory location. Therefore Paging is used.

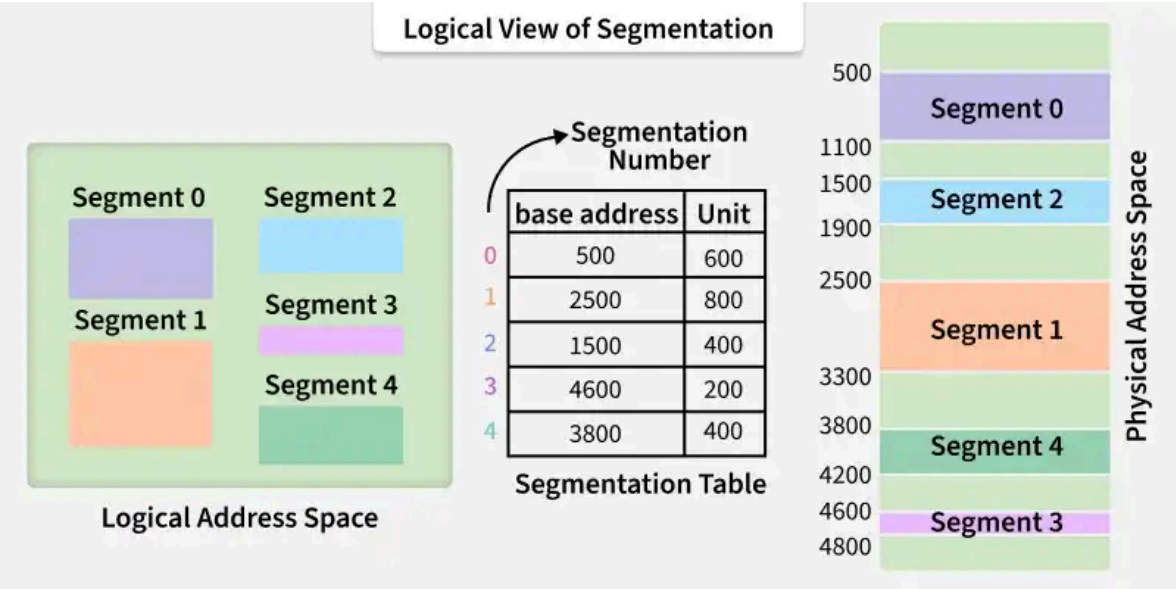
Paging

In paging, physical memory is divided into fixed-size frames and virtual memory is divided into fixed-size pages. The size of a page is equal to the size of a frame.



Segmentation

Segmentation is implemented to give users view of memory. The logical address space is a collection of segments. Segmentation can be implemented with or without the use of paging.



Page Fault: A page fault is a type of interrupt, raised by the hardware when a running program accesses a memory page that is mapped into the virtual address space, but not loaded in main/virtual memory.

Page Replacement Algorithms

1. First In First Out (FIFO)

This is the simplest page replacement algorithm. In this algorithm, operating system keeps track of all pages in the memory in a queue, oldest page is in the front of the queue. When a page needs to be replaced page in the front of the queue is selected for removal.

For example, consider page reference string 1, 3, 0, 3, 5, 6 and 3 page slots.

- Initially, all slots are empty, so when 1, 3, 0 came they are allocated to the empty slots —> 3 Page Faults.
- When 3 comes, it is already in memory so —> 0 Page Faults. Then 5 comes, it is not available in memory so it replaces the oldest page slot i.e 1. —> 1 Page Fault.
- Finally, 6 comes, it is also not available in memory so it replaces the oldest page slot i.e 3 —> 6 Page Fault.

Belady's anomaly: Belady's anomaly proves that it is possible to have more page faults when increasing the number of page frames while using the First in First Out (FIFO) page replacement algorithm.

For example, if we consider reference string 3 2 1 0 3 2 4 3 2 1 0 4 and 3 slots, we get 9 total page faults, but if we increase slots to 4, we get 10 page faults.

2. Optimal Page replacement

In this algorithm, pages are replaced which are not used for the longest duration of time in the future.

Let us consider page reference string 7 0 1 2 0 3 0 4 2 3 0 3 2 and 4 page slots.

Initially, all slots are empty, so when 7 0 1 2 are allocated to the empty slots —> 4 Page faults.
0 is already there so —> 0 Page fault.
When 3 came it will take the place of 7 because it is not used for the longest duration of time in the future.—> 1 Page fault.
0 is already there so —> 0 Page fault.
4 will takes place of 1 —> 1 Page Fault.
Now for the further page reference string —> 0 Page fault because they are already available in the memory.

Optimal page replacement is perfect, but not possible in practice as an operating system cannot know future requests. The use of Optimal Page replacement is to set up a benchmark so that other replacement algorithms can be analyzed against it.

3. Least Recently Used (LRU)

In this algorithm, the page will be replaced which is least recently used.

Let say the page reference string 7 0 1 2 0 3 0 4 2 3 0 3 2 .

Initially, we have 4-page slots empty. Initially, all slots are empty, so when 7 0 1 2 are allocated to the empty slots —> 4 Page faults.
0 is already their so —> 0 Page fault.
When 3 came it will take the place of 7 because it is least recently used —> 1 Page fault.
0 is already in memory so —> 0 Page fault. 4 will takes place of 1 —> 1 Page Fault.
Now for the further page reference string —> 0 Page fault because they are already available in the memory.

4. Most Recently Used (MRU)

In this algorithm, page will be replaced which has been used recently. Belady's anomaly can occur in this algorithm.

Example: Consider the page reference string 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3 with 4-page frames. Find number of page faults using MRU Page Replacement Algorithm.

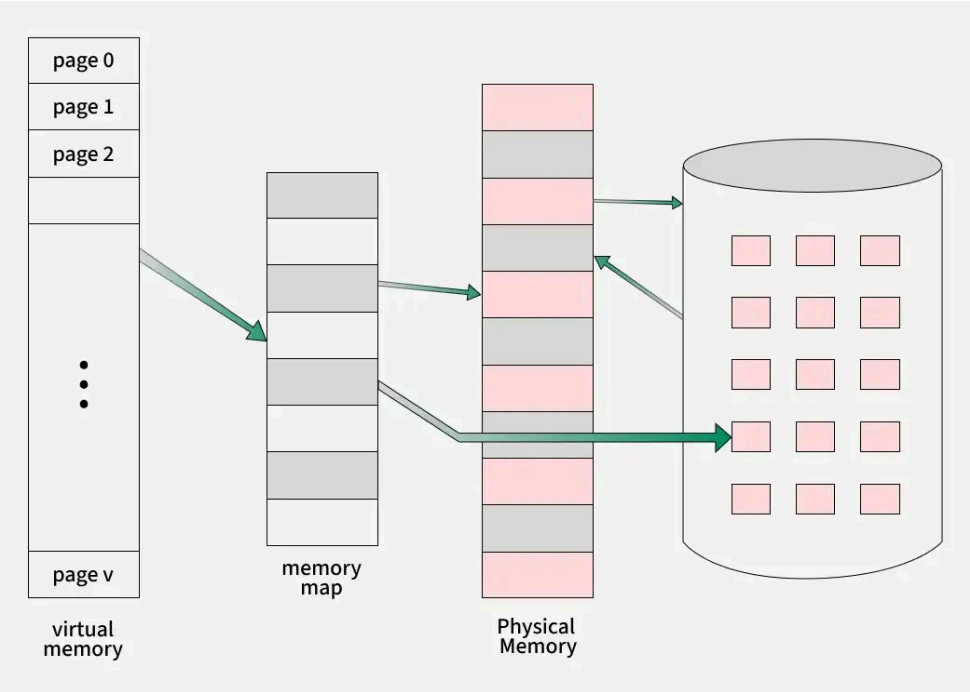
- *Initially, all slots are empty, so when 7 0 1 2 are allocated to the empty slots —> **4 Page faults***
- *0 is already their so–> **0 page fault***
- *when 3 comes it will take place of 0 because it is most recently used —> **1 Page fault***
- *when 0 comes it will take place of 3 —> **1 Page fault***
- *when 4 comes it will take place of 0 —> **1 Page fault***
- *2 is already in memory so —> **0 Page fault***
- *when 3 comes it will take place of 2 —> **1 Page fault***
- *when 0 comes it will take place of 3 —> **1 Page fault***
- *when 3 comes it will take place of 0 —> **1 Page fault***
- *when 2 comes it will take place of 3 —> **1 Page fault***
- *when 3 comes it will take place of 2 —> **1 Page fault***

Virtual Memory

Virtual memory is a memory management technique used by operating systems to give the appearance of a large, continuous block of memory to applications, even if the physical memory (RAM) is limited. It allows larger applications to run on systems with less RAM.

Key Points

- Uses secondary storage (disk) as an extension of RAM
- Allows execution of large programs
- Improves memory utilization
- Based on paging and demand paging



Demand Paging

Demand paging is a memory management technique used in operating systems where a page (a fixed-size block of memory) is only loaded into the computer's RAM when it is needed, or "demanded" by a process.

How Demand Paging Works:

1. **When a Program Starts:** Initially, only a part of the program (some pages) is loaded into RAM. The rest of the program is stored on the hard drive (secondary memory).
2. **When a Page is Needed:** If the program needs a page that is not currently in RAM, a **page fault** occurs. This means the operating system must fetch the required page from the hard drive and load it into RAM.
3. **Efficient Memory Use:** With demand paging, only the parts of the program that are actively used are in memory, which makes better use of RAM. This allows the system to run large programs without needing all of their data loaded into memory at once.

Advantages

- Better RAM utilization
- Faster program startup
- Supports virtual memory efficiently

Thrashing

Thrashing is a situation in which the operating system spends more time swapping data between RAM (main memory) and the hard drive (secondary memory) than actually executing processes. This happens when there isn't enough physical memory (RAM) to handle the current workload, causing the system to constantly swap pages in and out of memory.

Cause

- Too many processes
- Insufficient RAM
- High page fault rate

Effects

- CPU utilization drops
- System becomes very slow

Techniques to Handle Thrashing

- Reduce degree of multiprogramming
- Use working set model
- Increase physical memory
- Better page replacement algorithms

File Systems

A file system is a method an operating system uses to store, organize, and manage files and directories on a storage device.

File Directories

The collection of files is a file directory. The directory contains information about the files, including attributes, location, and ownership. Much of this information, especially that is concerned with storage, is managed by the operating system.

Below are information contained in a device directory.

- Name
- Type
- Address
- Current length
- Maximum length
- Date last accessed
- Date last updated
- Owner id
- Protection information

The operation performed on the directory are:

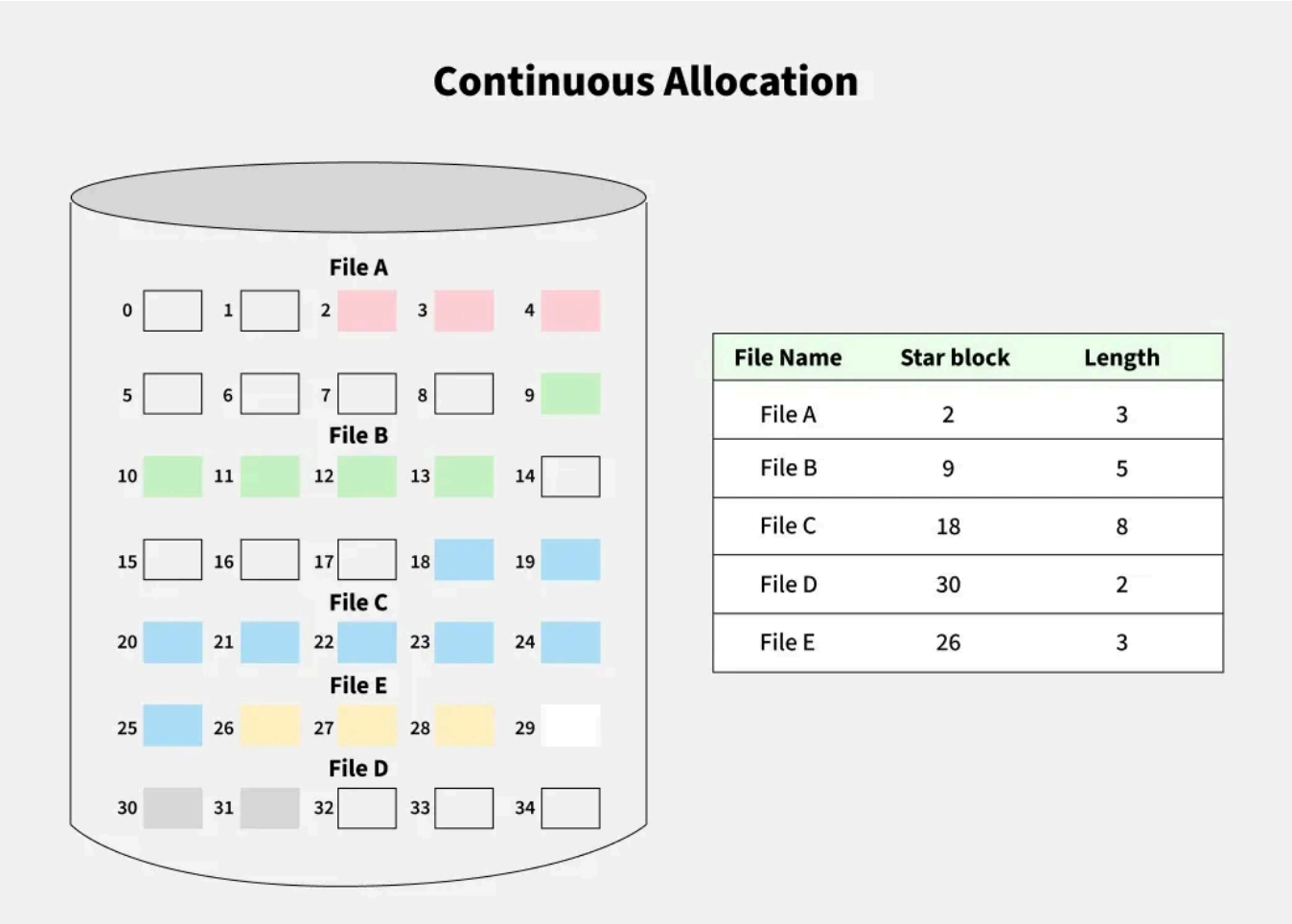
- Search for a file
- Create a file
- Delete a file
- List a directory
- Rename a file

- Traverse the file system

File Allocation Methods

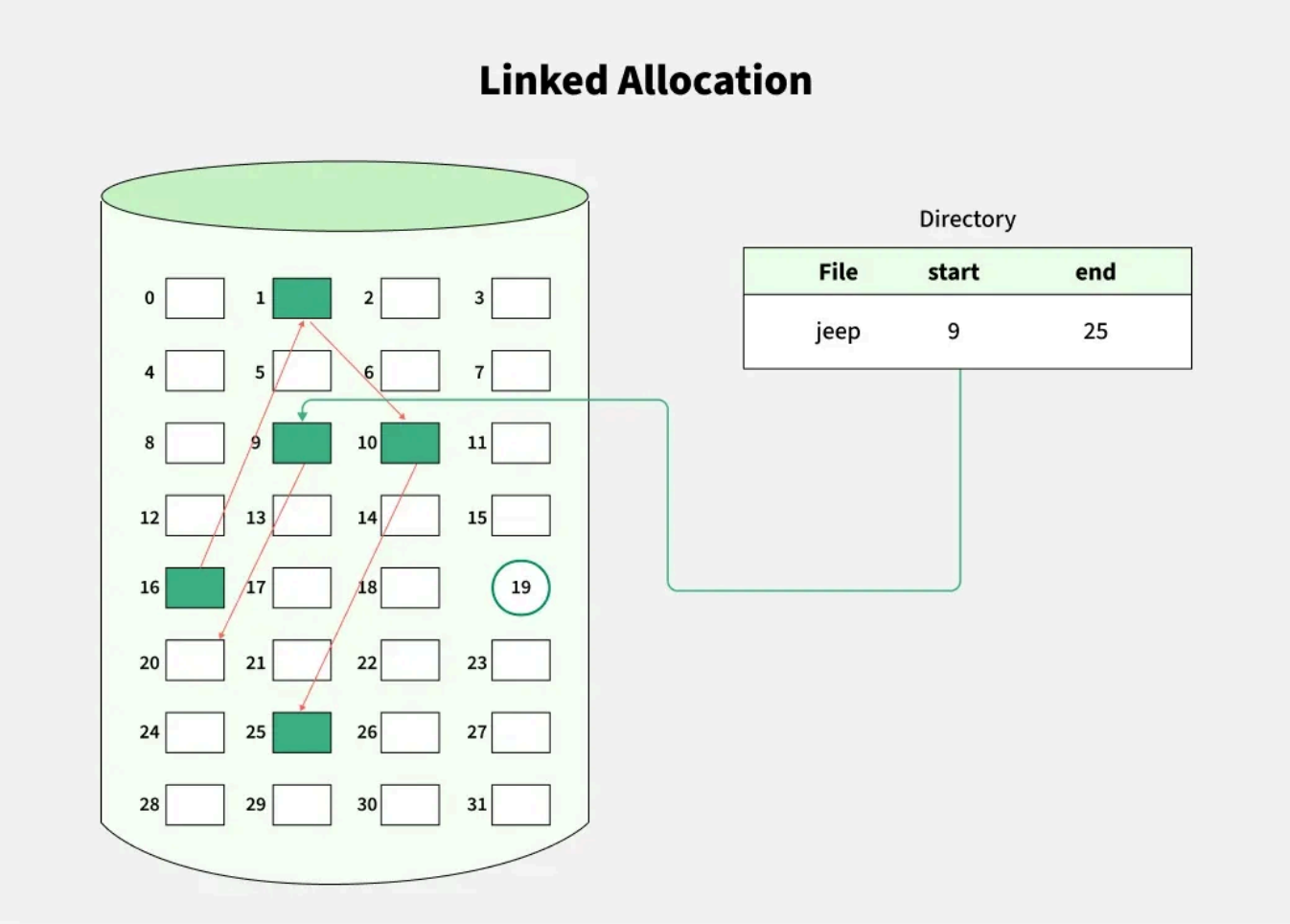
There are several types of file allocation methods. These are mentioned below.

Continuous Allocation: A single continuous set of blocks is allocated to a file at the time of file creation. Thus, this is a pre-allocation strategy, using variable size portions. The file allocation table needs just a single entry for each file, showing the starting block and the length of the file.

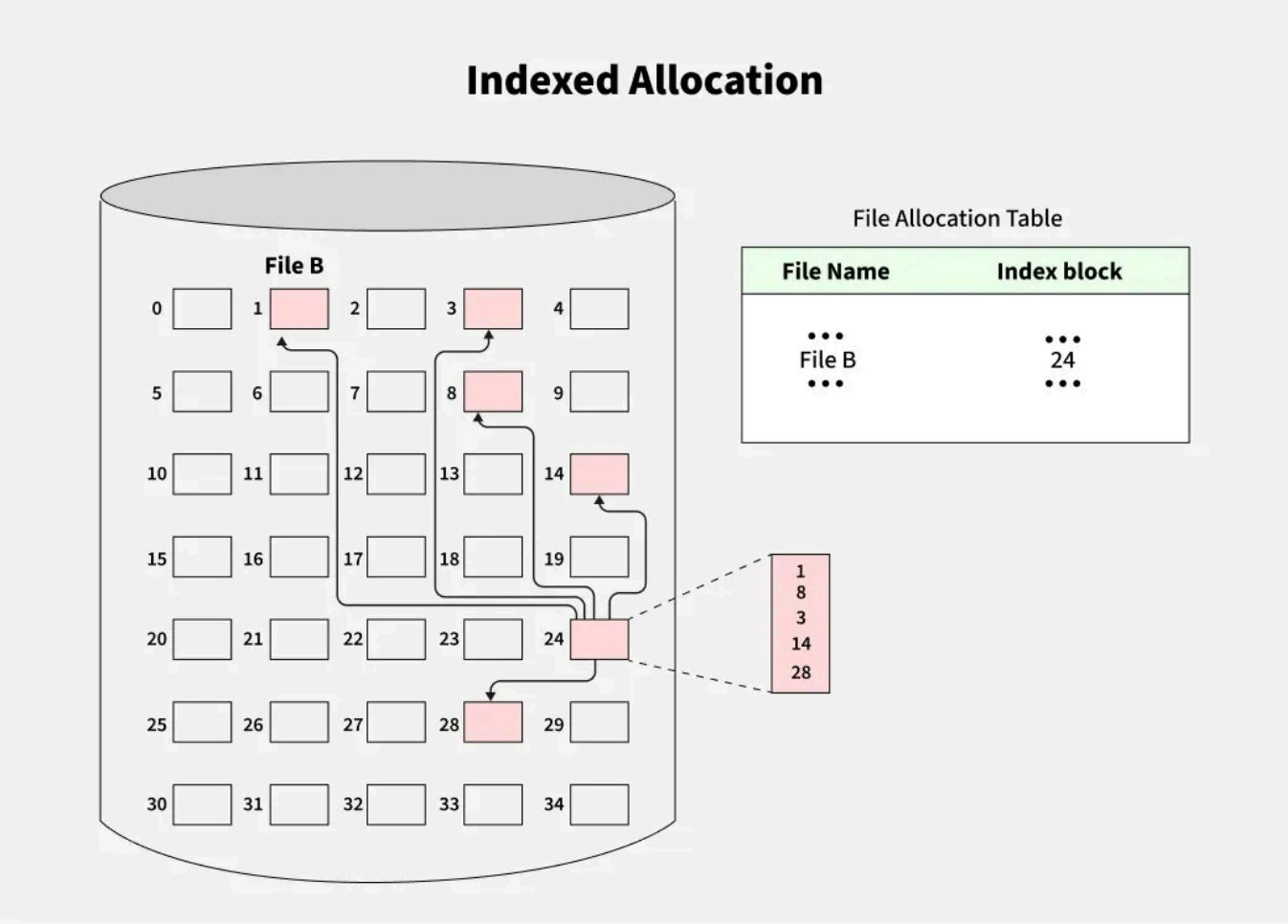


File Name	Star block	Length
File A	2	3
File B	9	5
File C	18	8
File D	30	2
File E	26	3

Linked Allocation(Non-contiguous allocation): Allocation is on an individual block basis. Each block contains a pointer to the next block in the chain. Again the file table needs just a single entry for each file, showing the starting block and the length of the file. Although pre-allocation is possible, it is more common simply to allocate blocks as needed. Any free block can be added to the chain.



Indexed Allocation: Indexed Allocation allocates disk blocks non-contiguously. A separate index block is used to store the addresses of all the data blocks of a file. The file table contains the address of the index block. Data blocks do **not** point to each other.



Disk Scheduling

Disk scheduling algorithms decide which disk request to service next to reduce seek time and improve performance.

Disk Scheduling Algorithms

FCFS (First Come First Serve)

- Requests served in arrival order
- Simple but poor performance

SSTF (Shortest Seek Time First)

- Serves request closest to disk head
- Reduces average seek time
- May cause starvation

SCAN (Elevator Algorithm)

- Disk head moves in one direction
- Services requests on the way
- Reverses direction at end

C-SCAN (Circular SCAN)

- Head moves in one direction only
- After reaching end, jumps to start
- Provides uniform waiting time

LOOK

- Similar to SCAN
- Head moves only till last request
- Avoids unnecessary movement

C-LOOK

- Circular version of LOOK
- Moves to last request, then jumps back

LIFO (Last In First Out)

- Latest request served first
- Can cause starvation

Algorithm	Advantages	Disadvantages
First Come First Served	• Easy to implement • Fair chance to all the I/O requests • It doesn't suffer from starvation	• Seek time is not optimized. • It doesn't maximize throughput.
Shortest Seek Time First	• It tends to minimize the arm movement. • It has better throughput than FCFS.	• It may suffer from starvation.
SCAN/LOOK	• It eliminates starvation. • It works well with light to heavy loads	• It is complex to implement • Increased overhead • It needs a directional bit to keep track of the head direction.
C-SCAN/C-LOOK	• No directional bit is required • It works well with light to heavy loads.	• It is complex to implement. • Increased overhead.

Comparison Among the Disk Scheduling Algorithms

Comment K kartik + Follow 792

Article Tags: Operating Systems GATE CS

GeeksforGeeks Sanchhaya Education Private Limited

Corporate & Communications Address:
A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:
K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305

Company: About Us, Legal, Privacy Policy, Careers, Contact Us, Corporate Solution, Campus Training

Explore: POTD, Practice Problems, Connect, Blogs, Upskill Courses

Tutorials: Programming Languages, DSA, Web Technology, AI, ML & Data Science, DevOps, CS Core Subjects, GATE, School Subjects, Software and Tools

Courses: ML and Data Science, DSA and Placements, Web Development, Data Science, Programming Languages, DevOps & Cloud, GATE, Trending Technologies

Offline Centers: Noida, Bengaluru, Pune, Hyderabad, Kolkata

Preparation Corner: Interview Corner, Aptitude, Puzzles, GfG 160, System Design

GET IT ON Google Play, Download on the App Store

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved